

Introduction

This report documents the design and implementation of a technical analysis toolkit for a cryptocurrency exchange platform. The program processes raw CSV market data, computes analytical metrics, and allows users to interact with an order book simulation.

TASK 1: Loading Trading Data and Computing OHLC Data

(i) Task Description The objective of this task was to compute Open-High-Low-Close (OHLC) data from raw order book entries and present it in a summary table. The user needed the ability to specify a product (e.g., ETH/USDT) and filter the results by a specific date range. To achieve this, a custom OHLCEntry struct was introduced to store daily summaries. The OrderBook class was updated to group chronologically sorted orders by their date substring, extracting the first and last prices for the Open and Close, and computing the maximum and minimum prices for the High and Low.

(ii) Implementation Logic & Pseudocode The implementation uses a `std::map<string, vector>` to group orders. As the CSV data is inherently chronological, extracting the Open and Close prices is efficiently achieved by accessing the `.front()` and `.back()` elements of each day's vector.

Pseudocode:

Plaintext FUNCTION `getOHLCData(type, product, startDate, endDate)`: CREATE map `ordersByDate`

```
FOR EACH order IN orders:
    IF order.type == type AND order.product == product:
        EXTRACT date FROM order.timestamp (first 10 characters)

        IF date >= startDate AND date <= endDate:
            APPEND order TO ordersByDate[date]

CREATE list ohlcData

FOR EACH (date, dailyOrders) IN ordersByDate:
    open = dailyOrders.FIRST().price
    close = dailyOrders.LAST().price
    high = computeMaxPrice(dailyOrders)
    low = computeMinPrice(dailyOrders)

    APPEND {date, open, high, low, close} TO ohlcData

RETURN ohlcData
```

(iii) Final Output (Take a screenshot of the terminal running option 7, entering a product like ETH/BTC, and displaying the formatted OHLC table).

TASK 2: Market Statistics and Safety Validations

(i) Task Description This task involved parsing the entire dataset to identify all known products and computing the highest and lowest available asks/bids for the current timeframe. A critical part of this task was implementing robust memory safety validations. If an empty vector was passed to the maximum or minimum price calculators, it would cause a segmentation fault. Safety guards were implemented to return a default value (0.0) if no orders were present for a specific product in a given timeframe.

(ii) Implementation Logic & Pseudocode To find known products, the system iterates through the order book and uses a `std::map<string, bool>` to automatically filter out duplicates. For the high/low computations, the algorithm assumes the first element is the target, then iterates through the remaining elements, updating the target if a strictly higher (or lower) value is found.

```
FUNCTION getHighPrice(orders):
  IF orders IS EMPTY:
    RETURN 0.0 // Prevent Segmentation Fault

    maxPrice = orders[0].price

  FOR EACH order IN orders:
    IF order.price > maxPrice:
      maxPrice = order.price

  RETURN maxPrice
```

(iii) Final Output (Take a screenshot of the terminal running option 2, successfully displaying the products, asks seen, max ask, and min ask for the current timeframe).

TASK 3: Robust User Order Entry (Bids & Asks)

(i) Task Description This task required capturing user input to generate new bids or asks and integrating them into the system. The input string (e.g., ETH/BTC,200,0.5) needed to be tokenized and safely converted into floating-point numbers. A major challenge addressed here was input buffer management; leftover newline characters (`\n`) from previous integer inputs caused the string capture (`getline`) to be skipped. This was resolved by explicitly clearing the input buffer using `.`

(ii) Implementation Logic & Pseudocode The program reads the user's string, splits it by commas using the `CSVReader::tokenise` function, and validates that exactly three tokens (product, price, amount) were provided. The strings are then passed through a try-catch block via `stod()` to ensure they are valid numbers before constructing a new `OrderBookEntry`.

```
FUNCTION enterAsk():
  PRINT "Enter product, price, amount"
  READ input line from user

  tokens = split(input, delimiter=',')

  IF tokens.length != 3:
    PRINT "Bad input"
    RETURN
```

```

TRY:
    price = convertToDouble(tokens[1])
    amount = convertToDouble(tokens[2])
    CREATE OrderBookEntry(price, amount, currentTime, tokens[0], ASK)
CATCH:
    PRINT "Invalid numbers provided"

```

(iii) Final Output (Take a screenshot of the terminal running option 3, typing in a valid format like "ETH/BTC,200,0.5", and the program acknowledging the input).

TASK 4: Timeframe Progression Simulation

(i) Task Description To simulate a live trading environment, the toolkit must allow the user to step forward through time. The program processes all orders sequentially based on their timestamps. This task involved creating a function to scan the dataset, find the current timestamp, and locate the strictly next chronological timestamp to advance the simulation state.

(ii) Implementation Logic & Pseudocode The algorithm iterates through the full list of orders. Since the CSV is sorted chronologically, the first timestamp it encounters that is strictly greater (>) than the currentTime is identified as the next timeframe. If the loop finishes without finding a greater timestamp (meaning the end of the file was reached), the system loops back to the earliest known timestamp (orders[0].timestamp) to keep the simulation running continuously.

```

FUNCTION getNextTime(currentTimestamp):
    nextTimestamp = ""

    FOR EACH order IN orders:
        IF order.timestamp > currentTimestamp:
            nextTimestamp = order.timestamp
            BREAK loop

    IF nextTimestamp == "":
        // Reached the end of the dataset, loop back to start
        nextTimestamp = orders[0].timestamp

    RETURN nextTimestamp

```

(iii) Final Output (Take a screenshot showing the main menu, running option 6 to go to the next timeframe, and then the main menu re-printing with the successfully updated timestamp).